

Chapter 6: Graph Neural Networks (GNNs) in Chemistry

Chapter 2 represented a molecule three ways — a SMILES string, a fixed hashed fingerprint, and, briefly, the underlying graph itself (Section 2.3), noting that a fingerprint is “a fast, lossy fixed-size summary” of that graph and that “learning similarity directly from $(\mathbf{A}, \mathbf{X})$ with graph neural networks... Chapter 6 covers in full.” This is that chapter. Chapter 5’s classical models (Random Forest, SVM, XGBoost) all consumed a Chapter 2 fingerprint as a fixed input vector; every model in this chapter instead consumes the molecular graph directly, learning its own task-specific representation rather than having one fixed in advance.

6.1 Motivation for GNNs

A Morgan/ECFP4 fingerprint (Chapter 2, Section 2.2) is built by a fixed, non-learned algorithm: hash each atom’s local environment out to some radius, fold the results into a bit vector, done. The procedure never sees a training label, so it cannot allocate representational capacity toward whichever structural patterns actually matter for the property being predicted — every bit is spent identically regardless of whether the downstream task is solubility, toxicity, or binding affinity. Two concrete costs follow directly from this, both named explicitly in Chapter 2: hashing **collisions**, where chemically distinct environments alias to the same bit as the dictionary of possible environments competes for a fixed-width vector, and a hard **radius cutoff**, beyond which no information propagates no matter how relevant a distant substructure is to the property in question.

Duvenaud et al. (2015) framed the fix precisely: replace the fixed hashing step with a differentiable, learnable analogue of the same circular-fingerprint procedure — at each layer, aggregate information from each atom’s immediate neighbors, exactly as Morgan’s algorithm does, but with a neural network instead of a hash function, trained end-to-end against the actual prediction target. Stacking L such layers lets information reach L bonds away, precisely as increasing an ECFP’s radius does, but where a fingerprint discards everything except the final hashed identifier, a GNN keeps every intermediate representation differentiable and lets gradient descent decide what those L rounds of neighborhood aggregation should actually encode for the task at hand. This is the core idea this chapter formalizes: **spatial neighborhood aggregation** — often called **message passing** — as a learnable generalization of the fixed circular fingerprints Chapter 2 built by hand.

Formally, recall Chapter 2’s notation (Section 2.3): a molecule is a graph $G = (V, E)$ with a node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ (one row per atom) and an adjacency structure $\mathbf{A}$ recording which atoms are bonded. Every architecture in this chapter computes a sequence of updated node representations $\mathbf{H}^{(0)} = \mathbf{X}, \mathbf{H}^{(1)}, \dots, \mathbf{H}^{(L)}$, where $\mathbf{H}^{(l)}$ is produced from $\mathbf{H}^{(l-1)}$ and $\mathbf{A}$ by exactly one round of neighborhood aggregation — an operation this chapter now defines precisely, in three increasingly general forms.

6.2 GCN & GAT Architectures

Graph Convolutional Networks (GCN; Kipf & Welling, 2017) derive their update rule as a first-order approximation of spectral convolution on a graph, which reduces, per layer, to:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with self-loops added (so a node’s own previous representation contributes to its update, not only its neighbors’), $\tilde{\mathbf{D}}$ is $\tilde{\mathbf{A}}$ ’s degree matrix, $\mathbf{W}^{(l)}$ is a learned weight matrix shared across every node in the graph, and σ is a nonlinearity (ReLU throughout this chapter’s implementation). The $\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ term is a symmetric normalization: it averages each node’s neighbors rather than summing them raw, so a highly-connected atom (a ring fusion carbon) does not automatically dominate a sparsely-connected one (a terminal methyl) purely by virtue of degree. Every node’s update uses the *same* $\mathbf{W}^{(l)}$, which is what makes the layer’s parameter count independent of molecule size and lets one trained model apply to molecules of any size.

Graph Attention Networks (GAT; Veličković et al., 2018) replace the fixed, degree-based normalization above with *learned, asymmetric* attention weights: rather than deciding a priori that every neighbor should be averaged equally, GAT learns how much each neighbor should contribute to a given atom’s update, conditioned on both atoms’ current features. For neighbor j of atom i :

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^\top[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))}, \quad \mathbf{h}_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j^{(l)}\right)$$

where $\parallel$ denotes concatenation, $\mathbf{a}$ and $\mathbf{W}$ are learned, and $\mathcal{N}(i)$ is atom i ’s bonded neighborhood. The softmax over $\mathcal{N}(i)$ guarantees the attention weights for a given atom sum to 1, so this is still a weighted average of neighbor features — but the weights themselves are now a learned function of the two atoms’ features, letting the model up-weight a chemically important neighbor (say, a nearby heteroatom) and down-weight an unremarkable one, rather than treating every bonded neighbor identically as plain GCN does. In practice, GAT layers use several independent attention heads in parallel (each with its own $\mathbf{a}$, $\mathbf{W}$) and concatenate or average their outputs — this chapter’s implementation uses 4 heads, each producing `hidden_dim/4` features, concatenated back to `hidden_dim`.

Neither formulation above uses **edge features** — a GCN layer only sees which atoms are bonded, a GAT layer only sees the two bonded atoms’ own features, and neither consumes bond order, conjugation, or stereochemistry directly in its aggregation step (both architectures were originally developed for citation-network graphs, where “edges” carry no intrinsic feature beyond their existence). For a chemical graph, where bond type is often exactly the information that distinguishes an aromatic ring from a saturated one, that is a real limitation — and it is precisely what motivates Section 6.3’s architecture.

6.3 Message Passing Neural Networks (MPNN)

Gilmer et al. (2017) unify GCN, GAT, and a wide family of earlier graph-convolutional architectures — including Duvenaud et al.’s (2015) learnable fingerprint from Section 6.1 — under a single framework, **Message Passing Neural Networks**, defined by two learned functions applied for L rounds:

$$\mathbf{m}_i^{(l+1)} = \sum_{j \in \mathcal{N}(i)} M_l(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}, \mathbf{e}_{ij}), \quad \mathbf{h}_i^{(l+1)} = U_l(\mathbf{h}_i^{(l)}, \mathbf{m}_i^{(l+1)})$$

where M_l (the **message function**) computes a message from every bonded neighbor j using both atoms’ current representations *and* the bond features $\mathbf{e}_{ij}$ connecting them, the messages are aggregated (summed, here) into $\mathbf{m}_i^{(l+1)}$, and U_l (the **update function**) combines the aggregated message with atom i ’s own previous representation to produce its new one. GCN and GAT are both special cases: GCN’s M_l discards $\mathbf{e}_{ij}$ entirely and uses a fixed-degree-normalized weight; GAT’s M_l discards $\mathbf{e}_{ij}$ too but replaces the fixed weight with a learned attention coefficient. What makes this framework strictly more general — and the reason it is the architecture Section 6.5’s hands-on project builds — is that M_l can condition on $\mathbf{e}_{ij}$ directly, so bond order, conjugation, and stereochemistry participate in the aggregation instead of being invisible to it.

This chapter’s implementation uses Gilmer et al.’s **edge network** variant of M_l specifically, which PyTorch Geometric implements directly as `NNConv`: the message function is itself a small neural network, $M_l(\mathbf{h}_i, \mathbf{h}_j, \mathbf{e}_{ij}) = \mathbf{A}_l(\mathbf{e}_{ij}) \mathbf{h}_j$, where $\mathbf{A}_l$ is a learned function (a 2-layer MLP in this chapter’s code) mapping a bond’s features to an entire `hidden_dim` $\times$ `hidden_dim` weight matrix, applied to the neighbor’s representation. In other words, every distinct bond environment generates its *own* linear transformation of the message passed along it, rather than every bond in the graph sharing one fixed transform — concretely, an aromatic C=C bond and a saturated C-C bond propagate information differently, because the edge network produces a different weight matrix for each. The update function U_l in this chapter’s implementation is a plain residual-free ReLU ($\mathbf{h}_i^{(l+1)} = \sigma(\mathbf{m}_i^{(l+1)})$), matching PyG’s default `NNConv` behavior when composed with an activation) rather than the gated recurrent unit (GRU) Gilmer et al.’s original paper uses for U_l , and this chapter’s readout is global mean pooling rather than their `Set2Set` readout — both are explicit simplifications, named here rather than silently substituted, made because the edge-conditioned message function is the architecturally distinctive contribution this section is teaching, and the simpler update/readout choices keep Section 6.5’s implementation legible without changing that core mechanism.

6.4 3D Equivariant Networks

Every architecture so far operates purely on the molecular graph’s *topology* — which atoms are bonded to which — with no reference to 3D geometry, echoing Chapter 2’s point (Section 2.4) that a graph fixes constitution but not shape. Many of the properties later chapters care about most — binding pose energetics (Chapter 11), force-field-level potential energy (Chapter 12) — are fundamentally properties of a specific 3D arrangement of atoms in space, not of the bond graph alone, which is what motivates a class of architectures that take atomic coordinates $\mathbf{r}_i \in \mathbb{R}^3$ as input alongside (or instead of) bond connectivity.

Building such an architecture naively is subtly wrong. A molecule’s physical properties do not depend on how it happens to be rotated, translated, or reflected in the coordinate frame a dataset happens to store it in — a solubility or a binding energy is the same number whether the input coordinates were rotated 90° first or not — but a generic neural network consuming raw (x, y, z) coordinates has no reason to respect that fact and will, in general, output different predictions

for the identical molecule presented in a different orientation. **Invariance** and **equivariance** are the two precise properties an architecture can guarantee against this failure mode. For a transformation g (a rotation and/or translation, from the group $E(3)$ of rigid Euclidean motions, or its rotation-only subgroup $SE(3)$) acting on the input coordinates, a function f is:

- **Invariant** if $f(g \cdot \mathbf{r}) = f(\mathbf{r})$ for every g — the output is unchanged by the transformation. A predicted scalar property (solubility, an energy) *should* be invariant: it is a single number, and there is no meaningful sense in which rotating the input should change it.
- **Equivariant** if $f(g \cdot \mathbf{r}) = g \cdot f(\mathbf{r})$ — the output transforms *the same way* the input did, rather than staying fixed. A predicted force or velocity vector, which has its own direction in space, *should* be equivariant: rotate the molecule, and the predicted force vectors must rotate identically, or the prediction is physically inconsistent with itself.

A network with either property gets that consistency guaranteed by its architecture rather than approximated by hoping enough rotated training examples were seen — which matters directly for **data efficiency**: an invariant or equivariant model does not need to learn the same physics separately for every orientation a molecule might appear in during training, because the constraint is structural, not learned. Three concrete architectures illustrate the design space:

- **SchNet** (Schütt et al., 2018) is *invariant*: it represents each pairwise atomic distance $\|\mathbf{r}_i - \mathbf{r}_j\|$ (itself already rotation- and translation-invariant, since a distance between two points does not change under a rigid motion) through continuous, learned radial filters, and aggregates only scalar quantities throughout — so nothing equivariant ever needs to be tracked, at the cost of being unable to output a genuinely directional quantity like a force.
- **EGNN** (Satorras et al., 2021) is *equivariant*: alongside scalar node features, it maintains and updates the atomic coordinates themselves at every layer, using update rules built specifically so that rotating the input coordinates produces an identically rotated set of output coordinates, without requiring the heavier spherical-harmonic machinery some equivariant architectures use.
- **NequIP** (Batzner et al., 2022) is *equivariant* via that heavier route: it builds features that transform according to the irreducible representations of $O(3)$ (using spherical harmonics and tensor products), giving it a strictly larger space of equivariant functions to represent higher-order directional information — at real computational cost, in exchange for state-of-the-art data efficiency on interatomic-potential tasks (directly relevant to Chapter 12’s neural network potentials, which this same paper targets).

Section 6.5’s hands-on project does not implement a 3D-aware architecture: both ESOL and FreeSolv, as distributed through MoleculeNet, are 2D graphs with no bundled 3D conformer (generating one would require Chapter 2, Section 2.4’s ETKDG step, applied per molecule, which this chapter does not do), so GCN, GAT, and this chapter’s MPNN all operate on topology alone, consistent with Sections 6.1-6.3. This section is deliberately theory-only for that reason — the architectures above become directly load-bearing once 3D structure is actually on the table, starting with Chapter 9’s structure prediction and, especially, Chapter 12’s neural network potentials.

6.5 Hands-on Project: GNN Molecular Property Prediction

The project code lives in this chapter’s folder (ch06_gnn_molecular_properties/) and implements Sections 6.2-6.3’s architectures directly in PyTorch Geometric (Fey & Lenssen, 2019), built on PyTorch (Paszke et al., 2019), to predict two real physicochemical properties from the molecular graph alone.

A note on the second property’s name

The outline this book follows describes this project as predicting “aqueous solubility and binding free energy.” The solubility half is exact: **ESOL** (Delaney, 2004) is 1,128 compounds’ measured aqueous solubility, expressed as log-molar solubility. The second dataset this project actually uses, however — **FreeSolv** (Mobley & Guthrie, 2014), 642 compounds — measures **hydration free energy**: the free energy change of transferring a molecule from vacuum into water. That is a real, well-defined, and widely used thermodynamic property, and it is the standard MoleculeNet regression benchmark conventionally paired with ESOL in exactly this kind of exercise — but it is not the same quantity as **protein-ligand binding free energy** (the free energy of a ligand binding a specific protein pocket), which depends on a specific 3D protein structure and is the subject of Chapter 11’s docking methods, not this chapter’s. Rather than mislabel FreeSolv’s actual property to match the outline’s phrase, or invent a protein-ligand dataset this chapter’s 2D-graph architectures are not equipped to use correctly (per Section 6.4, that would need explicit 3D structural input), this section uses FreeSolv under its real name and states the distinction plainly here.

Featurization and models

Both datasets are loaded through PyTorch Geometric’s built-in MoleculeNet class, which parses each compound’s SMILES with RDKit and featurizes it using the categorical atom/bond scheme introduced by the Open Graph Benchmark (Hu et al., 2020): 9 categorical features per atom (atomic number, chirality, degree, formal charge, attached-hydrogen count, radical electron count, hybridization, aromaticity, ring membership) and 3 per bond (bond type, stereochemistry, conjugation). gnn_property.py’s AtomEncoder and BondEncoder turn these categorical indices into dense vectors by summing one learned embedding per feature — a standard, simple encoding scheme that keeps every downstream layer working with ordinary continuous vectors:

```
class AtomEncoder(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.embeddings = nn.ModuleList(
            nn.Embedding(dim, hidden_dim) for dim in NODE_FEATURE_DIMS
        )

    def forward(self, x):
        return sum(emb(x[:, i]) for i, emb in enumerate(self.embeddings))
```

GCNRegressor, GATRegressor, and MPNNRegressor each stack 3 message-passing layers (GCNConv, GATConv, and NNConv respectively — Sections 6.2-6.3) of width 64 over these encoded atom features, mean-pool the final atom representations into one graph-level vector (global_mean_pool),

and pass that through a small 2-layer MLP to a single scalar prediction. MPNNRegressor additionally runs every bond’s encoded features through BondEncoder and into each NNConv layer’s edge network, exactly as Section 6.3 describes.

Splitting, training, and evaluation

gnn_property.py reimplements Chapter 5’s scaffold_split (Bemis & Murcko, 1996) for this regression setting — group compounds by generic ring scaffold, then greedily fill the training set scaffold-group by scaffold-group until the target training fraction is reached — alongside a stratification-free random split, so the same random-vs-scaffold comparison Chapter 5 demonstrated for hERG classification can be repeated here for property regression. Targets are standardized (zero mean, unit variance, computed from the training split only) before training and un-standardized before computing metrics, a routine but necessary step for stable regression training that has no analogue in Chapter 5’s classification setting. Models train for 60 epochs with the Adam optimizer (learning rate 10^{-3}) minimizing mean squared error on the standardized target, evaluated on the held-out test set by root-mean-squared error (RMSE) and R^2 , both in the property’s original units.

Results: GCN vs. GAT vs. MPNN on ESOL

Training all three architectures on ESOL with an identical protocol (60 epochs, hidden dimension 64, 3 layers, seed 0) gives:

Model	Split	RMSE (log mol/L)	MAE	R^2
GCN	random	1.011	0.789	0.787
GCN	scaffold	1.246	0.968	0.635
GAT	random	0.794	0.626	0.869
GAT	scaffold	1.229	0.937	0.645
MPNN	random	0.809	0.626	0.864
MPNN	scaffold	1.537	1.196	0.445

Two findings, both real and both worth stating plainly rather than rounding off to fit a tidier narrative. First, the pattern from Chapter 5 repeats exactly: every architecture’s R^2 drops substantially under scaffold split relative to random split (GCN: 0.787 $\rightarrow$ 0.635; GAT: 0.869 $\rightarrow$ 0.645; MPNN: 0.864 $\rightarrow$ 0.445) — random splitting overestimates real-world generalization for graph-structured molecular data regardless of which specific architecture consumes the graph, which is exactly the general lesson Chapter 5 established with fingerprints and classical models. Second, and less tidy: **MPNN, the architecturally most expressive of the three, generalizes worst under scaffold split** — worse than the simpler GCN and GAT, despite matching or beating them under random split. The edge-conditioned message function (NNConv’s edge network) gives MPNN substantially more parameters per layer than GCN’s shared weight matrix or GAT’s attention vector, on a training set of only 902 molecules; more capacity without more data is a standard recipe for overfitting, and scaffold split — by construction — is the evaluation that most exposes overfitting to the training set’s specific chemical series. This is not evidence that MPNNs are generally worse

than GCNs for molecular property prediction (Section 6.3’s citations, and the wider literature, show the opposite in larger-data settings); it is evidence that architectural expressiveness and generalization under distribution shift are separate axes, and that a fair benchmark has to check both rather than assume the more sophisticated model wins by default.

Results: MPNN on FreeSolv

Running the same MPNN, unmodified, on FreeSolv (this project’s second required property, per the note above):

Split	RMSE (kcal/mol)	MAE	R^2
random	1.612	1.127	0.845
scaffold	4.272	3.220	0.304

The random-vs-scaffold gap is even more pronounced here than on ESOL — RMSE nearly triples (1.61 $\rightarrow$ 4.27 kcal/mol) and R^2 falls from 0.845 to 0.304. FreeSolv is both smaller (642 compounds total, so a scaffold split leaves fewer training examples per scaffold group to learn from) and chemically broader in origin — it was assembled for the SAMPL blind prediction challenges specifically to stress-test free-energy methods across diverse chemotypes (Mobley & Guthrie, 2014) — both of which make genuine cross-scaffold generalization harder than on ESOL’s more homogeneous compound set, consistent with the larger gap observed here.

Reproducibility

Dependencies are version-floored (torch $\geq$ 2.2, torch_geometric $\geq$ 2.5, rdkit $\geq$ 2023.9.1 in requirements.txt, validated against torch 2.13.0 and torch_geometric 2.8.0.post1). This chapter’s data/delaney-processed.csv and data/SAMPL.csv are the real, unmodified raw MoleculeNet CSVs for ESOL and FreeSolv, bundled so --use-cached-raw seeds PyTorch Geometric’s dataset cache directly instead of downloading from the live DeepChem S3 bucket — the same resilience pattern Chapter 4 established. The 21-test suite in tests/test_gnn_property.py checks the atom/bond encoders, all three model architectures’ forward and backward passes, both split strategies (including an explicit no-scaffold-leakage check), and end-to-end training against a small synthetic molecule set and a real fixture subset, running in seconds; one test calls the live MoleculeNet download directly. pip install -r requirements-dev.txt && pytest reproduces all 21 results. The full 60-epoch runs behind the tables above are slower (roughly 2-3 minutes per split on one CPU core) and are not run in the test suite for that reason — reproduce them directly with python gnn_property.py --use-cached-raw --dataset esol --model mpnn --split both, substituting --dataset freesolv for the second table.

Limitations and what comes next

Everything this chapter’s project measures is still, ultimately, an average over one training run at one fixed random seed — the tables above report a single seed (0) throughout, for direct comparability across models and splits, not because run-to-run variance is known to be small; a careful benchmark would repeat each configuration across several seeds and report a spread, which this chapter’s code supports (--seed) but its reported tables do not exercise. More funda-

mentally, every architecture in this chapter, including the hands-on project’s MPNN, operates on 2D molecular topology alone — Section 6.4’s equivariant architectures exist precisely because many properties later chapters care about are not topology-alone properties at all. Chapter 7’s generative models build directly on the graph representations this chapter introduced, now used to *produce* novel molecular graphs rather than only score existing ones; Chapters 8 and 9 extend graph- and attention-based architectures from small molecules to proteins, at far larger scale; and Chapter 12’s neural network potentials are, architecturally, direct descendants of Section 6.4’s equivariant networks, applied to the 3D, dynamics-aware regime this chapter’s project deliberately did not enter.

A note on Google Colab

Colab’s default runtime preinstalls torch but not torch_geometric or rdkit; run `!pip install torch_geometric rdkit` in the first cell. A GPU runtime accelerates training automatically if selected but is not required — every result in this chapter was produced on CPU.

References

- Duvenaud, D., Maclaurin, D., Aguilera-Iparraguirre, J., Gómez-Bombarelli, R., Hirzel, T., Aspuru-Guzik, A., & Adams, R. P. (2015). Convolutional networks on graphs for learning molecular fingerprints. *Advances in Neural Information Processing Systems*, 28 (NeurIPS 2015). arXiv:1509.09292 (no DOI; NeurIPS proceedings papers from this era were not assigned one).
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *5th International Conference on Learning Representations* (ICLR 2017). arXiv:1609.02907 (no DOI; ICLR, via OpenReview, does not assign one).
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. *6th International Conference on Learning Representations* (ICLR 2018). arXiv:1710.10903 (no DOI, as above).
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70, 1263-1272 (no DOI; PMLR does not assign one).
- Schütt, K. T., Sauceda, H. E., Kindermans, P.-J., Tkatchenko, A., & Müller, K.-R. (2018). SchNet - A deep learning architecture for molecules and materials. *The Journal of Chemical Physics*, 148(24), 241722. <https://doi.org/10.1063/1.5019779>
- Satorras, V. G., Hoogeboom, E., & Welling, M. (2021). E(n) equivariant graph neural networks. *Proceedings of the 38th International Conference on Machine Learning*, PMLR 139, 9323-9332 (no DOI, as above).
- Batzner, S., Musaelian, A., Sun, L., Geiger, M., Mailoa, J. P., Kornbluth, M., Molinari, N., Smidt, T. E., & Kozinsky, B. (2022). E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications*, 13(1), 2453. <https://doi.org/10.1038/s41467-022-29939-5>
- Delaney, J. S. (2004). ESOL: Estimating aqueous solubility directly from molecular structure. *Journal of Chemical Information and Computer Sciences*, 44(3), 1000-1005. <https://doi.org/10.1021/ci034243x>

- Mobley, D. L., & Guthrie, J. P. (2014). FreeSolv: a database of experimental and calculated hydration free energies, with input files. *Journal of Computer-Aided Molecular Design*, 28(7), 711-720. <https://doi.org/10.1007/s10822-014-9747-x>
- Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., & Leskovec, J. (2020). Open graph benchmark: Datasets for machine learning on graphs. *Advances in Neural Information Processing Systems*, 33 (NeurIPS 2020). arXiv:2005.00687 (no DOI, as above).
- Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. *ICLR 2019 Workshop on Representation Learning on Graphs and Manifolds*. arXiv:1903.02428 (no DOI, as above).
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32 (NeurIPS 2019). arXiv:1912.01703 (no DOI, as above).

See Chapter 4’s references for Wu et al. (2018, MoleculeNet, the benchmark suite ESOL and FreeSolv are distributed through) and Bemis & Murcko (1996, scaffold definition, reused from Chapter 5) — both reused here rather than re-listed. Every citation above lacking a DOI was checked directly against its own official landing page (arXiv abstract page or the PMLR proceedings page) rather than assumed absent; ICLR (via OpenReview), PMLR, and NeurIPS proceedings from these years routinely do not assign DOIs, matching the same situation already documented for the Pedregosa et al. (2011) scikit-learn citation in Chapter 5’s references.

All dataset sizes and model metrics cited in Section 6.5 were computed directly by running `gnn_property.py` against the bundled ESOL/FreeSolv fixtures on 2026-08-20, not taken from a secondary source or from published benchmark leaderboards — see `data/delaney-processed.csv`, `data/SAMPL.csv`, and `gnn_property.py` to reproduce.